VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI – 590 018



Mini Project Report

on

"Interactive Activation Function Analysis"

Submitted in partial fulfilment of the requirements for the Award of degree of

Bachelor of Engineering in CSE(AI&ML)

Submitted By

Vishal S Narayankar 1RN23CI183
Yashas Kaustav 1RN23CI186

Under the Guidance of

Prof. Jayashree Nagaraj

Assistant Professor



Department of Computer Science And Engineering(AI&ML)

RNS INSTITUTE OF TECHNOLOGY

Autonomous Institution affiliated to Visvesvaraya Technological University, Belagavi

Approved by AICTE, New Delhi. Accredited by NAAC 'A+' Grade

Channasandra, Dr. Vishnuvardhan Road, Bengaluru - 560 098

Ph:(080)28611880,28611881 URL: www.rnsit.ac.in

RN SHETTY TRUST®

RNS INSTITUTE OF TECHNOLOGY

Autonomous Institution Affiliated to Visvesvaraya Technological University, Belagavi
Approved by AICTE, New Delhi, Accredited by NAAC with 'A+' Grade
RR Nagar, Dr. Vishnuvardhan Road, Bengaluru - 560 098 Ph:
(080) 28611880, 28611881 URL: www.rnsit.ac.in

DEPARTMENT OF CSE (AI&ML)



CERTIFICATE

Certified that the mini-project work entitled "Interactive Activation Function Analysis" carried out by Vishal S Narayankar [1RN23CI183], Yashas Kaustav [1RN23CI186] are Bonafide students of **RNS Institute of Technology** in partial fulfilment for the award of **"BACHELOR OF ENGINEERING"** in **CSE (AI&ML)** as prescribed by **VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI** during the academic year **2026 – 27**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the departmental library. The mini-project report has been approved as it satisfies the academic requirements in respect of mini-project work on DEEP LEARNING (BCA701) prescribed for the said degree.

Signature of Course Coordinator

Dept. of CSE (AI&ML)
RNSIT

ACKNOWLEDGEMENT

I want to profoundly thank the Management of **RNS Institute of Technology** for providing me such a healthy environment for pursuing and completing this project work.

I am thankful to **Dr. M K Venkatesha**, Director, RNS Institute of Technology, for providing me excellent facilities for carrying out this project.

I am grateful to **Dr. Ramesh Babu H S**, Principal, RNS Institute of Technology, for providing me excellent facilities and academic ambience for carrying out this project work at RNSIT.

I express my thanks to **Dr. Andhe Pallavi**, Prof. & Head, Dept. of CSE (AI & ML), for her valuable support and encouragement.

I sincerely acknowledge and thank the support, guidance, and encouragement extended by my project guide, **Prof. Jayashree Nagaraj**, Assistant Professor, Dept. of CSE(AI&ML). This project would not have been possible without their valuable insights and timely suggestions.

Finally, I take this opportunity to extend my gratitude and respect to my parents, teaching and non-teaching staff of the Department, and all my friends who have directly or indirectly supported me during the period of my project.

Team Member 1: Vishal N
USN: 1RN23CI183

Team Member 2: Yashas K
USN: 1RN23CI186

TABLE OF CONTENTS

1. Introduction..... 1

2. Problem Description..... 2

3. Aim of the Project..... 2

4. Literature Survey 2

5. Objectives 4

6. Methodology..... 5

7. Technologies..... 6

8. Functional Requirements..... 6

9. Expected Outcomes..... 7

10. Results 8

11. Future Enhancements 9

12. Conclusion 10

13. References 10

TABLE OF FIGURES

1. Results 8

Interactive Activation Function Analysis

This project presents an interactive application for studying and comparing five activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU, and GELU in feed-forward neural networks. Developed using Python, PyTorch, and Streamlit, the application allows users to train neural networks on synthetic regression tasks and visualise their predictions against theoretical function curves. Mean Squared Error (MSE) and training loss graphs are used to monitor model performance and analyse learning behaviour across epochs. The application enables controlled comparisons of activation functions under consistent training conditions, helping users understand their characteristics, advantages, and limitations. It also demonstrates important concepts such as vanishing gradients in Sigmoid and Tanh and the potential for inactive neurons in ReLU. Overall, the project aims to bridge the gap between theoretical concepts and practical experimentation through interactive visualisation and reproducible analysis.

Introduction

Artificial Neural Networks (ANNs) are fundamental to modern artificial intelligence and deep learning. Activation functions introduce non-linearity into neural networks, enabling them to learn complex patterns and relationships. Their selection influences training efficiency, convergence, and gradient propagation.

This project presents an interactive application for studying five activation functions: Sigmoid, Hyperbolic Tangent (Tanh), Rectified Linear Unit (ReLU), Leaky ReLU, and Gaussian Error Linear Unit (GELU). Each function exhibits distinct characteristics, including vanishing gradients in Sigmoid and Tanh and the dying-ReLU problem in standard ReLU.

Developed using Python, PyTorch, and Streamlit, the application enables users to train feed-forward neural networks on mathematically defined functions and visualise their predictions against theoretical curves. Mean squared error (MSE) graphs illustrate training progress. A comparative mode evaluates all five activation functions under controlled experimental conditions.

The project aims to bridge the gap between theoretical understanding and practical experimentation through interactive visualisation. It also includes a deep-network diagnostic module designed to investigate vanishing gradients and inactive ReLU neurons, which requires further integration before experimental evaluation.

Problem Description

Understanding activation functions is a challenge for students learning deep learning, as their effects on neural network training and gradient propagation are difficult to understand through theory alone. Sigmoid and Tanh may suffer from vanishing gradients, while ReLU can experience the dying-neuron problem. Leaky ReLU addresses this limitation, whereas GELU provides a smooth nonlinear transformation. This project develops an interactive application using Python, PyTorch, and Streamlit to demonstrate and compare these five activation functions. Using synthetic regression tasks, the system visualises predicted outputs against theoretical curves and tracks training performance through mean squared error (MSE). A controlled comparison enables users to observe differences under consistent training conditions. The objective is to improve understanding through interactive visualisation and reproducible experimentation rather than identify a universally superior activation function.

Aim of the Project

The aim of this project is to develop an interactive application for studying and comparing five activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU, and GELU. The system uses deep learning techniques to train feed-forward neural networks and visualise their learning behaviour through predicted curves and mean squared error graphs. It enables users to experiment with different training parameters and observe how activation functions influence approximation accuracy and convergence. The project seeks to improve the understanding of activation functions through controlled experiments, interactive visualisation, and reproducible results. Additionally, it provides a foundation for investigating vanishing gradients and inactive ReLU neurons in deeper neural networks.

.

Literature Survey

Activation functions are essential components of artificial neural networks because they introduce non-linearity, enabling networks to learn complex relationships between inputs and outputs. The choice of activation function can influence gradient propagation, convergence speed, training stability, and overall learning behaviour. Several studies have investigated these characteristics and proposed activation functions and optimisation techniques to improve neural network training.

Without activation functions, a neural network composed of multiple layers would behave like a single linear transformation, limiting its ability to model complex patterns in data. Activation functions overcome this limitation by transforming the outputs of neurons and allowing networks to learn nonlinear relationships.

Different activation functions exhibit distinct mathematical properties, which affect how information and gradients propagate through the network during training.

Glorot and Bengio (2010) studied the challenges associated with training deep feed-forward neural networks and highlighted the limitations of sigmoid-based activation functions [1]. The sigmoid function maps input values to an output range between zero and one, making it useful in specific applications. However, its gradient becomes very small when the input values are highly positive or negative. This saturation can contribute to the vanishing-gradient problem, limiting the ability of earlier layers to learn effectively. Their research also emphasised the importance of weight initialisation in maintaining effective gradient propagation during training.

Nair and Hinton (2010) investigated the use of Rectified Linear Units (ReLU) in neural networks and demonstrated their potential for learning effective representations [2]. ReLU returns zero for negative inputs and the input itself for positive values. Compared with sigmoid-based functions, it does not saturate for positive inputs, which can help gradients propagate more effectively through a network. Its computational simplicity also makes it practical for neural network training. However, ReLU has a limitation known as the dying-ReLU problem, in which a neuron may consistently produce zero outputs and stop learning when its inputs remain negative.

He et al. (2015) addressed the challenges of training deep networks using rectifier-based activation functions by proposing a weight-initialisation method designed to maintain signal variance across layers [3]. Their work demonstrated the importance of considering weight initialisation when designing and training neural networks. Although activation functions influence gradient behaviour, their effectiveness can also depend on network architecture, initial parameter values, and optimisation settings. This finding is relevant to comparative experiments because differences in training behaviour should be interpreted in the context of the complete training configuration.

Hendrycks and Gimpel (2016) introduced the Gaussian Error Linear Unit (GELU), a smooth activation function that scales inputs according to the cumulative distribution function of a standard normal distribution [4]. Unlike ReLU, which applies a sharp threshold at zero, GELU provides a smoother transition between negative and positive inputs. This characteristic makes it an important alternative for studying nonlinear transformations in neural networks. Comparing GELU with ReLU and Leaky ReLU helps illustrate how differences in activation-function shape may affect learning behaviour.

Kingma and Ba (2015) proposed Adam, an optimisation algorithm that adapts learning rates for individual model parameters using estimates of first and second moments of gradients [6]. Adam is widely used in neural network training because it combines adaptive learning rates with momentum-like behaviour. In the present project, the use of Adam provides a consistent optimisation approach for comparing activation

functions. However, the choice of optimiser and its hyperparameters can influence training outcomes and should therefore remain controlled during comparisons.

LeCun et al. discussed the MNIST handwritten-digit dataset, which has become a widely used benchmark for evaluating machine-learning and neural-network methods [5]. Although the present project focuses on synthetic regression tasks, established datasets such as MNIST provide a foundation for extending the investigation to classification problems in future work.

Building on these studies, the present project develops an interactive application for comparing Sigmoid, Tanh, ReLU, Leaky ReLU, and GELU in feed-forward neural networks. Using Python, PyTorch, and Streamlit, the application enables users to train models on synthetic regression functions, visualise predicted outputs against theoretical curves, and examine training loss using mean squared error.

Objectives

- To implement five activation functions: Sigmoid, Hyperbolic Tangent (Tanh), Rectified Linear Unit (ReLU), Leaky ReLU, and Gaussian Error Linear Unit (GELU).
- To develop an interactive application using Python, PyTorch, and Streamlit that enables users to experiment with different activation functions.
- To train feed-forward neural networks on synthetic regression tasks and observe their learning behaviour.
- To visualise the predicted outputs of neural networks against theoretical function curves to evaluate how accurately each model learns the target relationship.
- To monitor and compare training performance using Mean Squared Error (MSE) and loss-versus-epoch graphs.
- To provide a controlled comparison of activation functions by maintaining consistent training conditions, allowing their learning behaviour to be examined under comparable settings.
- To demonstrate the theoretical characteristics and limitations of different activation functions, including vanishing gradients in Sigmoid and Tanh and the dying-ReLU problem.
- To improve the conceptual understanding of activation functions through interactive visualisation and reproducible experimentation, providing a foundation for further investigation into neural network training and optimisation.

Methodology

The project follows an experimental approach to study and compare five activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU, and GELU. The application is developed using Python, PyTorch, and Streamlit to provide an interactive environment for training neural networks and visualising their learning behaviour.

The five activation functions are implemented using PyTorch and integrated into feed-forward neural networks. Synthetic regression datasets are generated from mathematically defined functions, providing known target outputs against which model predictions can be evaluated. The neural networks are trained to approximate these functions, allowing users to observe how different activation functions influence the learning process.

Model training is performed using the Adam optimisation algorithm, with Mean Squared Error (MSE) serving as the loss function to measure the difference between predicted and target values. Training performance is monitored across epochs to examine changes in prediction error. Consistent experimental conditions are maintained during comparative analysis to make the observed differences more meaningful.

The application visualises the predicted outputs alongside the theoretical function curves, enabling users to assess how accurately each network approximates the target function. Loss-versus-epoch graphs are also provided to illustrate training progress and compare the learning behaviour of different activation functions. These visualisations help users understand differences in convergence and model performance across the selected regression tasks.

A Streamlit-based interface allows users to select activation functions, configure experiments, initiate model training, and observe the resulting graphs. This interactive approach connects the theoretical properties of activation functions with their practical behaviour during neural network training.

The project focuses on experimental observation rather than identifying a universally superior activation function. Although a deep-network diagnostic module is intended to investigate vanishing gradients and inactive ReLU neurons, it requires further integration before its results can be evaluated.

Technologies

- **Python** : The primary programming language used to implement activation functions, generate datasets, and conduct experiments.
- **PyTorch** : A deep learning framework used to build and train feed-forward neural networks, implement activation functions, and calculate gradients.
- **Streamlit** : Used to develop an interactive web interface for selecting activation functions, configuring experiments, and displaying results.
- **NumPy** : Used for numerical computations and generating synthetic regression datasets.
- **Matplotlib** : Used to visualise theoretical function curves, model predictions, and training loss graphs.
- **Adam Optimiser** : Used to update network parameters during training and improve model convergence.
- **Mean Squared Error (MSE**) : Used as the loss function to measure the difference between predicted and actual values.

Functional Requirements

- Activation Function Selection: The system shall allow users to select and experiment with Sigmoid, Tanh, ReLU, Leaky ReLU, and GELU activation functions.
- Dataset Generation: The system shall generate synthetic regression datasets using mathematically defined functions.
- Neural Network Training: The system shall train feed-forward neural networks using the selected activation function.
- Parameter Configuration: The system shall allow users to configure relevant training parameters for conducting experiments.
- Prediction Visualisation: The system shall display the model's predicted outputs alongside the theoretical function curves.
- Loss Calculation: The system shall calculate Mean Squared Error (MSE) to evaluate the difference between predicted and target values.

- Training Progress Monitoring: The system shall display loss-versus-epoch graphs to illustrate changes in training error.
- Comparative Analysis: The system shall support comparisons between activation functions under consistent experimental conditions.
- Interactive Interface: The system shall provide a Streamlit-based interface through which users can configure experiments, initiate training, and view results.
- Result Presentation: The system shall present training results and visualisations in an understandable format to support the study of activation-function behaviour.

Expected Outcomes

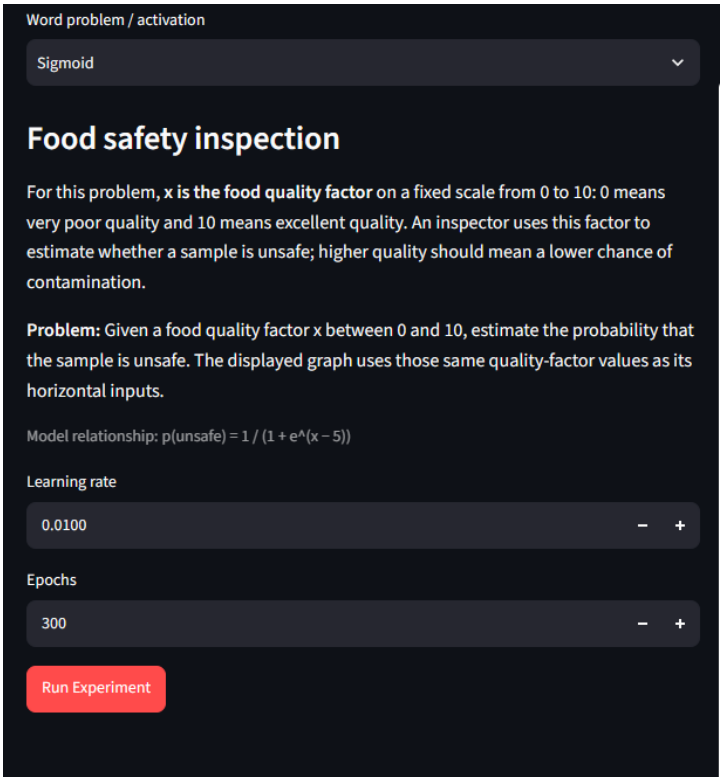
The project is expected to provide an interactive platform for understanding and comparing five activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU, and GELU. By training feed-forward neural networks on synthetic regression tasks, users will be able to observe how different activation functions influence model predictions and learning behaviour.

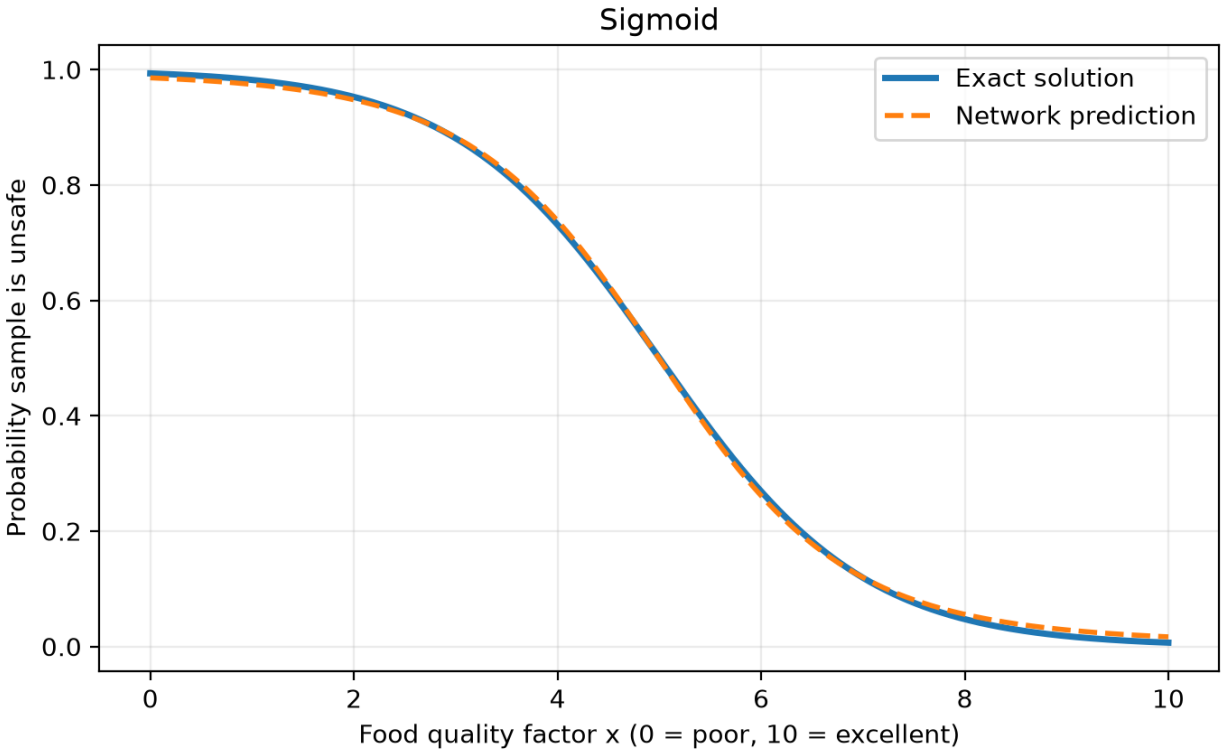
The application is expected to generate visual comparisons between predicted outputs and theoretical function curves, helping users evaluate how effectively each model learns a given relationship. Training loss graphs based on Mean Squared Error (MSE) will provide insights into changes in prediction error over successive epochs.

The project is also expected to demonstrate the theoretical characteristics of different activation functions, including the saturation behaviour of Sigmoid and Tanh and the potential for inactive neurons in ReLU. Comparing these functions under consistent experimental conditions will help users understand their advantages, limitations, and differences in learning behaviour.

Furthermore, the Streamlit interface is expected to make experimentation accessible through interactive controls and visualisations. Overall, the project aims to bridge the gap between theoretical knowledge and practical implementation, providing an educational foundation for understanding activation functions and their influence on feed-forward neural network training.

Results


Word problem / activation
Sigmoid
Food safety inspection
For this problem, x is the food quality factor on a fixed scale from 0 to 10: 0 means very poor quality and 10 means excellent quality. An inspector uses this factor to estimate whether a sample is unsafe; higher quality should mean a lower chance of contamination.
Problem: Given a food quality factor x between 0 and 10, estimate the probability that the sample is unsafe. The displayed graph uses those same quality-factor values as its horizontal inputs.
Model relationship: p(unsafe) = 1 / (1 + e^(x − 5))
Learning rate
0.0100
Epochs
300
Run Experiment


Sigmoid
Exact solution
Network prediction
Probability sample is unsafe
1.0
0.8
0.6
0.4
0.2
0.0
0
2
4
6
8
10
Food quality factor x (0 = poor, 10 = excellent)

Future Enhancements

The project can be extended by integrating the deep-network diagnostic module to investigate vanishing gradients and inactive ReLU neurons across multiple hidden layers. This would provide a deeper understanding of how activation functions influence gradient propagation and training stability in deeper neural networks.

The application can also be expanded to support additional activation functions, such as Swish, Mish, and ELU, enabling broader comparative analysis. Furthermore, experiments could be extended beyond synthetic regression tasks to classification problems using standard datasets such as MNIST.

Additional evaluation metrics, including accuracy for classification tasks and other regression metrics, could be incorporated to provide a more comprehensive assessment of model performance. The application could also support configurable network architectures, including variations in the number of hidden layers and neurons, allowing users to examine how network design affects learning behaviour.

Finally, the interface could be improved with interactive mathematical visualisations of activation functions and their derivatives. These enhancements would make the application more informative and versatile, providing a stronger educational platform for studying activation functions and their role in neural network training.

Conclusion

This project presents an interactive application for studying and comparing five activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU, and GELU in feed-forward neural networks. Developed using Python, PyTorch, and Streamlit, the application enables users to train models on synthetic regression tasks and visualise their predictions against theoretical function curves.

The use of Mean Squared Error (MSE) and training loss graphs provides a practical approach to examining model learning behaviour and changes in prediction error. By comparing activation functions under consistent experimental conditions, the application helps users understand their mathematical properties, advantages, and limitations. It also highlights important concepts such as vanishing gradients in Sigmoid and Tanh and the potential for inactive neurons in ReLU.

The project bridges the gap between theoretical concepts and practical experimentation through interactive visualisation.

References

[1] X. Glorot and Y. Bengio, "Understanding the Difficulty of Training Deep Feedforward Neural Networks," **Proceedings of the 13th International Conference on Artificial Intelligence and Statistics, vol. 9, pp. 249–256, 2010**. Available: https://proceedings.mlr.press/v9/glorot10a.html

[2] V. Nair and G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," **Proceedings of the 27th International Conference on Machine Learning, pp. 807–814, 2010**. Available: https://www.cs.toronto.edu/~fritz/absps/reluICML.pdf

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," **Proceedings of the IEEE International Conference on Computer Vision, pp. 1026–1034, 2015**. Available: https://doi.org/10.1109/ICCV.2015.123

[4] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv preprint **arXiv:1606.08415, 2016**. Available: https://doi.org/10.48550/arXiv.1606.08415

[5] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST Database of Handwritten Digits," 1998. Available: http://yann.lecun.com/exdb/mnist/

[6] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," **International Conference on Learning Representations, 2015**. Available: https://arxiv.org/abs/1412.6980

[7] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," **Advances in Neural Information Processing Systems, vol. 32, 2019**. Available: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html